

Assignment #4: Software Tests

Andrea Eiríksdóttir, Jóhannes Reykdal Einarsson, Sigursteinn Óli Þorsteinsson, Sölvi Santos

March 2026

1. Scope

This document presents the implemented test fixture and mock objects for one controller component in our system.

Controller under test:

- `FlightController`

Methods covered by the fixture:

`searchFlights (StringdepartureCode,StringarrivalCode,ZonedDateTimedate)`

`filterByDepartureTimeRange (List<Flight>inputFlights,ZonedDateTimestart,ZonedDateTimeend)`

2. Test Fixture (JUnit)

The test fixture is implemented in `src/test/java/airline/controllers/FlightControllerTests.java`.

It provides:

- clean setup with `@BeforeEach`
- clean teardown with `@AfterEach`
- injection of a mock storage dependency into the controller under test
- deterministic fixture data for repeatable tests

Example fixture setup:

Listing 1: Fixture setup and teardown (excerpt)

```
@BeforeEach
void setup() {
    mockFlightRepository = new MockFlightRepository();
    controller = new FlightController(mockFlightRepository);
    ...
    // test data objects created here (f1, f2, f3, f4)
}

@AfterEach
void cleanup() {
    controller = null;
}
```

```
mockFlightRepository = null;  
...  
}
```

3. Test Cases and Expected Results

3

Test method	Input category	Expected result
searchFlights_returnsMatchingFlight_whenRouteAndDateMatch	Success case	One matching flight returned.
searchFlights_returnsEmptyList_whenNoFlightsMatch	Unsuccessful search	Empty list returned.
searchFlights_returnsEmptyList_withEmptyRepositoryMock	Bonus mock behavior	Empty list returned when storage mock is empty.
searchFlights_throwsIllegalArgumentException_whenInputIsInvalid	Invalid input (departureCode = null)	IllegalArgumentException.
searchFlights_throwsIllegalArgumentException_whenArrivalCodeIsBlank	Invalid input (blank arrival code)	IllegalArgumentException.
searchFlights_throwsIllegalArgumentException_whenDateIsNull	Invalid input (date = null)	IllegalArgumentException.
searchFlights_throwsIllegalArgumentException_whenDateIsInPast	Invalid input (date in the past)	IllegalArgumentException.
filterByDepartureTimeRange_returnsFlightsInSelectedDayRange	Success case	Flights from selected start day through selected end day are returned.
filterByDepartureTimeRange_includesWholeDay_whenStartAndEndHaveSameDate	Edge boundary case	Same-day range includes all flights on that day.
filterByDepartureTimeRange_throwsIllegalArgumentException_whenStartDateAfterEndDate	Invalid range	IllegalArgumentException.
filterByDepartureTimeRange_throwsIllegalArgumentException_whenInputListIsNull	Invalid input (inputFlights = null)	IllegalArgumentException.
filterByDepartureTimeRange_throwsIllegalArgumentException_whenStartIsNull	Invalid input (start = null)	IllegalArgumentException.
filterByDepartureTimeRange_throwsIllegalArgumentException_whenEndIsNull	Invalid input (end = null)	IllegalArgumentException.

Test method	Input category	Expected result
<code>filterByDepartureTimeRange_throwsIllegalArgumentException_whenDatesInPast</code>	Invalid input (dates in the past)	<code>IllegalArgumentException</code> .
<code>filterByDepartureTimeRange_returnsWholeDay_whenStartEqualsEndDate</code>	Edge case (<code>start == end</code>)	Full selected day is included.

4. Why These Test Cases

These tests were selected to cover the controller contract in plausible scenarios:

- successful behavior with valid input
- unsuccessful behavior with valid input
- invalid input handling
- boundary and edge conditions for day-range filtering

This aligns with the requirement to test a controller method in multiple realistic situations.

5. Why This Coverage Is Sufficient

Coverage is sufficient for Assignment #4 because:

- it verifies controller input validation for obvious invalid cases
- it verifies expected output shape and content for successful and unsuccessful searches
- it verifies edge behavior for the tested day-range method
- tests focus on controller behavior, not on internal implementation details of storage

6. Mock Objects and Simulated Behavior

Primary mock:

- `MockFlightRepository`

Simulated behavior:

- returns preconfigured search results for `findByRouteAndDate`
- does not implement real repository/database search logic
- keeps unrelated interface methods unsupported to remain minimal

Bonus mock with different behavior:

- `EmptyFlightRepositoryMock`

Simulated behavior:

- always behaves as empty storage
- enables testing controller behavior under “no data available” conditions

7. Source Code References

- Test fixture: `src/test/java/airline/controllers/FlightControllerTests.java`
- Mock object: `src/test/java/airline/repository/MockFlightRepository.java`
- Bonus mock object: `src/test/java/airline/repository/EmptyFlightRepositoryMock.java`

Appendix A: Full Test Fixture Source

```
package airline.controllers;

import airline.model.Airplane;
import airline.model.Airport;
import airline.model.Flight;
import airline.model.FlightStatus;
import airline.repository.EmptyFlightRepositoryMock;
import airline.repository.MockFlightRepository;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.time.ZoneId;
import java.time.ZonedDateTime;
import java.util.List;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

/**
 * Test fixture for FlightController.
 * It verifies controller-level behavior using a mocked storage dependency.
 */
public class FlightControllerTests {
    private FlightController controller;
    private MockFlightRepository mockFlightRepository;
    private Flight f1;
    private Flight f2;
    private Flight f3;
    private Flight f4;
    private ZonedDateTime baseDate;

    // Creates a clean controller + mock setup and deterministic fixture data for each
    // test.
    @BeforeEach
    void setup() {
        mockFlightRepository = new MockFlightRepository();
        controller = new FlightController(mockFlightRepository);

        Airport kef = new Airport("KEF", "Keflavik", "Reykjanesbaer", "Iceland");
        Airport lhr = new Airport("LHR", "Heathrow", "London", "UK");
        Airport cph = new Airport("CPH", "Kastrup", "Copenhagen", "Denmark");
        Airplane plane = new Airplane("B737", 180);

        ZoneId zone = ZoneId.of("UTC");
        baseDate = ZonedDateTime.of(2026, 4, 1, 0, 0, 0, 0, zone);

        f1 = new Flight("FI100", baseDate.plusHours(10), baseDate.plusHours(13), 100.0,
            FlightStatus.SCHEDULED, 180, lhr, kef, plane);
        f2 = new Flight("FI101", baseDate.plusDays(1).plusHours(10), baseDate.plusDays
            (1).plusHours(13), 110.0,
            FlightStatus.SCHEDULED, 180, lhr, kef, plane);
        f3 = new Flight("FI200", baseDate.plusHours(11), baseDate.plusHours(14), 120.0,
            FlightStatus.SCHEDULED, 180, cph, kef, plane);
        f4 = new Flight("FI201", baseDate.plusHours(9), baseDate.plusHours(12), 130.0,
```

```

        FlightStatus.SCHEDULED, 180, cph, kef, plane);
    }

    // Clears references after each test so state does not leak between tests.
    @AfterEach
    void cleanup() {
        controller = null;
        mockFlightRepository = null;
        f1 = null;
        f2 = null;
        f3 = null;
        f4 = null;
        baseDate = null;
    }

    // Success case: valid route/date should return matching flights.
    @Test
    @DisplayName("searchFlights returns matching flight for route and date")
    void searchFlights_returnsMatchingFlight_whenRouteAndDateMatch() {
        mockFlightRepository.setRouteAndDateResponse(List.of(f1));

        List<Flight> result = controller.searchFlights("KEF", "LHR", baseDate.plusHours(12));

        assertEquals(1, result.size());
        assertEquals("FI100", result.get(0).getFlightNumber());
    }

    // Unsuccessful search: valid input with no match should return an empty list.
    @Test
    @DisplayName("searchFlights returns empty list when there is no match")
    void searchFlights_returnsEmptyList_whenNoFlightsMatch() {
        mockFlightRepository.setRouteAndDateResponse(List.of());

        List<Flight> result = controller.searchFlights("KEF", "LHR", baseDate.plusDays(2));

        assertEquals(0, result.size());
    }

    // Bonus behavior: with an empty-storage mock, search should still return an empty list.
    @Test
    @DisplayName("searchFlights returns empty list with empty repository mock")
    void searchFlights_returnsEmptyList_withEmptyRepositoryMock() {
        FlightController emptyController = new FlightController(new EmptyFlightRepositoryMock());
        List<Flight> result = emptyController.searchFlights("KEF", "LHR", baseDate.plusHours(12));

        assertEquals(0, result.size());
    }

    // Invalid input: null departure code should be rejected by controller validation.
    @Test
    @DisplayName("searchFlights throws for invalid input")
    void searchFlights_throwsIllegalArgumentException_whenInputIsInvalid() {

```

```

        assertThrows(IllegalArgumentException.class,
            () -> controller.searchFlights(null, "LHR", baseDate.plusHours(12)));
    }

    // Invalid input: blank arrival code should be rejected by controller validation.
    @Test
    @DisplayName("searchFlights throws when arrival code is blank")
    void searchFlights_throwsIllegalArgumentException_whenArrivalCodeIsBlank() {
        assertThrows(IllegalArgumentException.class,
            () -> controller.searchFlights("KEF", " ", baseDate.plusHours(12)));
    }

    // Invalid input: null date should be rejected by controller validation.
    @Test
    @DisplayName("searchFlights throws when date is null")
    void searchFlights_throwsIllegalArgumentException_whenDateIsNull() {
        assertThrows(IllegalArgumentException.class,
            () -> controller.searchFlights("KEF", "LHR", null));
    }

    // Invalid input: past date should be rejected by controller validation
    @Test
    @DisplayName("searchFlights throws when date is in past")
    void searchFlights_throwsIllegalArgumentException_whenDateIsInPast(){
        assertThrows(IllegalArgumentException.class, () -> controller.searchFlights("
            KEF", "LHR", baseDate.minusYears(1)));
    }

    // Day-range filtering interprets start/end as calendar days.
    @Test
    @DisplayName("filterByDepartureTimeRange returns flights in selected day range")
    void filterByDepartureTimeRange_returnsFlightsInSelectedDayRange() {
        List<Flight> input = List.of(f1, f2, f3, f4);
        ZonedDateTime start = baseDate.plusHours(9);
        ZonedDateTime end = baseDate.plusHours(12);

        List<Flight> result = controller.filterByDepartureTimeRange(input, start, end);

        assertEquals(3, result.size());
        assertEquals("FI100", result.get(0).getFlightNumber());
        assertEquals("FI200", result.get(1).getFlightNumber());
        assertEquals("FI201", result.get(2).getFlightNumber());
    }

    // Same-day range should include whole selected day even if times differ.
    @Test
    @DisplayName("filterByDepartureTimeRange includes whole day when start and end are
        same date")
    void filterByDepartureTimeRange_includesWholeDay_whenStartAndEndHaveSameDate() {
        List<Flight> input = List.of(f1, f3, f4);
        ZonedDateTime start = baseDate.plusHours(9);
        ZonedDateTime end = baseDate.plusHours(11);

        List<Flight> result = controller.filterByDepartureTimeRange(input, start, end);

        assertEquals(3, result.size());
        assertEquals("FI100", result.get(0).getFlightNumber());
        assertEquals("FI200", result.get(1).getFlightNumber());
    }

```



```

        assertEquals("FI201", result.get(2).getFlightNumber());
    }

    // Invalid range: start date after end date should throw an exception.
    @Test
    @DisplayName("filterByDepartureTimeRange throws when start date is after end date")
    void filterByDepartureTimeRange_throwsIllegalArgumentException_whenStartDateAfterEndDate() {
        List<Flight> input = List.of(f1, f3, f4);
        ZonedDateTime start = baseDate.plusDays(2).plusHours(12);
        ZonedDateTime end = baseDate.plusDays(1).plusHours(9);

        assertThrows(IllegalArgumentException.class,
            () -> controller.filterByDepartureTimeRange(input, start, end));
    }

    // Invalid input: null flight list should be rejected.
    @Test
    @DisplayName("filterByDepartureTimeRange throws when input list is null")
    void filterByDepartureTimeRange_throwsIllegalArgumentException_whenInputListIsNull() {
        assertThrows(IllegalArgumentException.class,
            () -> controller.filterByDepartureTimeRange(null, baseDate.plusHours(9),
                baseDate.plusHours(12)));
    }

    // Invalid input: null start time should be rejected.
    @Test
    @DisplayName("filterByDepartureTimeRange throws when start is null")
    void filterByDepartureTimeRange_throwsIllegalArgumentException_whenStartIsNull() {
        List<Flight> input = List.of(f1, f3, f4);

        assertThrows(IllegalArgumentException.class,
            () -> controller.filterByDepartureTimeRange(input, null, baseDate.
                plusHours(12)));
    }

    // Invalid input: null end time should be rejected.
    @Test
    @DisplayName("filterByDepartureTimeRange throws when end is null")
    void filterByDepartureTimeRange_throwsIllegalArgumentException_whenEndIsNull() {
        List<Flight> input = List.of(f1, f3, f4);

        assertThrows(IllegalArgumentException.class,
            () -> controller.filterByDepartureTimeRange(input, baseDate.plusHours
                (9), null));
    }

    // Invalid Input: dates in the past should be rejected
    @Test
    @DisplayName("filterByDepartureTimeRange throws when dates are in past")
    void filterByDepartureTimeRange_throwsIllegalArgumentException_whenDatesInPast() {
        List<Flight> input = List.of(f1, f3, f4);
        assertThrows(IllegalArgumentException.class,
            () -> controller.filterByDepartureTimeRange(input, baseDate.minusYears
                (2), baseDate.minusYears(1)));
    }

```

```

}

// Edge case: equal start/end date should include the entire selected day.
@Test
@DisplayName("filterByDepartureTimeRange returns whole day when start equals end
            date")
void filterByDepartureTimeRange_returnsWholeDay_whenStartEqualsEndDate() {
    List<Flight> input = List.of(f1, f2, f3, f4);
    ZonedDateTime pointInTime = baseDate.plusHours(10);

    List<Flight> result = controller.filterByDepartureTimeRange(input, pointInTime,
        pointInTime);

    assertEquals(3, result.size());
    assertEquals("FI100", result.get(0).getFlightNumber());
    assertEquals("FI200", result.get(1).getFlightNumber());
    assertEquals("FI201", result.get(2).getFlightNumber());
}
}

```

Appendix B: Full Mock Object Source

```
package airline.repository;

import airline.model.Flight;

import java.time.ZonedDateTime;
import java.util.ArrayList;
import java.util.List;

/**
 * Minimal mock implementation of {@link FlightRepository} for controller tests.
 *
 * <p>This mock does not implement real repository or database logic.
 * It only returns preconfigured values needed by tests.</p>
 */
public class MockFlightRepository implements FlightRepository {
    private List<Flight> routeAndDateResponse = new ArrayList<>();

    /**
     * Configures what findByRouteAndDate should return.
     *
     * @param flights flights to be returned by search calls
     */
    public void setRouteAndDateResponse(List<Flight> flights) {
        this.routeAndDateResponse = new ArrayList<>(flights);
    }

    /**
     * Returns preconfigured search response.
     * Parameters are intentionally ignored because this is a test stub.
     */
    @Override
    public List<Flight> findByRouteAndDate(String departureCode, String arrivalCode,
        ZonedDateTime date) {
        return new ArrayList<>(routeAndDateResponse);
    }

    @Override
    public List<Flight> findAll() {
        throw unsupported();
    }

    @Override
    public Flight findByFlightNumber(String flightNumber) {
        throw unsupported();
    }

    @Override
    public List<Flight> findByDepartureAirportAndDate(String airportCode,
        ZonedDateTime date) {
        throw unsupported();
    }

    @Override
    public List<Flight> findDepartingFlights(String airportCode) {
        throw unsupported();
    }
}
```

```

@Override
public List<Flight> findArrivingFlights(String airportCode) {
    throw unsupported();
}

@Override
public void save(Flight flight) {
    throw unsupported();
}

@Override
public void update(Flight flight) {
    throw unsupported();
}

@Override
public void delete(String flightNumber) {
    throw unsupported();
}

private UnsupportedOperationException unsupported() {
    return new UnsupportedOperationException("Not needed for current controller
        tests");
}
}

```

Appendix C: Bonus Mock Object Source

```
package airline.repository;

import airline.model.Flight;

import java.time.ZonedDateTime;
import java.util.ArrayList;
import java.util.List;

/**
 * A mock implementation of {@link FlightRepository} that simulates an
 * always-empty data source.
 *
 * <p>This mock is used in tests that verify behavior when no flights exist,
 * such as ensuring controllers return empty results or handle missing data
 * correctly.</p>
 *
 * <p>All query methods return empty collections (or {@code null} where applicable),
 * and all mutation methods ({@code save}, {@code update}, {@code delete})
 * are implemented as no-ops.</p>
 *
 * <p>This mock is intentionally stateless and ignores all inputs.</p>
 */
public class EmptyFlightRepositoryMock implements FlightRepository {

    @Override
    public List<Flight> findAll() {
        return new ArrayList<>();
    }

    @Override
    public Flight findByFlightNumber(String flightNumber) {
        return null;
    }

    @Override
    public List<Flight> findByRouteAndDate(String departureCode, String arrivalCode,
        ZonedDateTime date) {
        return new ArrayList<>();
    }

    @Override
    public List<Flight> findByDepartureAirportAndDate(String airportCode,
        ZonedDateTime date) {
        return new ArrayList<>();
    }

    @Override
    public List<Flight> findDepartingFlights(String airportCode) {
        return new ArrayList<>();
    }

    @Override
    public List<Flight> findArrivingFlights(String airportCode) {
        return new ArrayList<>();
    }
}
```

```
@Override
public void save(Flight flight) {
    // Intentionally no-op: this mock always simulates empty storage.
}

@Override
public void update(Flight flight) {
    // Intentionally no-op: this mock always simulates empty storage.
}

@Override
public void delete(String flightNumber) {
    // Intentionally no-op: this mock always simulates empty storage.
}
}
```